

UNIVERSIDAD TECNOLÓGICA DEL CENTRO DE VERACRUZ

IGS 9ª V-S

**Extracción del conocimiento de Bases de
Datos**

**Saber Hacer Producto:
Creación e Implementación de un Data
Warehouse usando PostgreSQL y Jupyter con
Python**

**Docente: MRySI. María Reina Zarate Nava
Alumno: Luis Alberto García Lezama**

ÍNDICE

INVESTIGACIÓN	3
¿Qué es un Data Warehouse?	3
Diferencias entre una Base de Datos Operacional y un Data Warehouse	4
Base de Datos Operacional (OLTP)	4
Data Warehouse (OLAP)	4
Ventajas del Uso de un Data Warehouse	5
Arquitecturas Comunes de un Data Warehouse	6
Herramientas y Tecnologías para Implementar un Data Warehouse	8
PRÁCTICA	10

INVESTIGACIÓN

¿Qué es un Data Warehouse?

Un Data Warehouse (Almacén de Datos) es un sistema diseñado para recopilar, integrar, almacenar y gestionar grandes volúmenes de información provenientes de diversas fuentes de datos. Su principal objetivo es facilitar el análisis de información histórica para apoyar la toma de decisiones estratégicas dentro de una organización.

A diferencia de las bases de datos transaccionales, que están orientadas a registrar operaciones diarias, un Data Warehouse está optimizado para consultas complejas, análisis de tendencias y generación de reportes.

Las principales características de un Data Warehouse son:

- Orientado a temas específicos como ventas, clientes o productos.
- Integrado, ya que reúne datos de múltiples fuentes.
- Histórico, porque conserva información de largos períodos de tiempo.
- No volátil, debido a que los datos almacenados rara vez son modificados o eliminados.

Diferencias entre una Base de Datos Operacional y un Data Warehouse

Las bases de datos operacionales y los Data Warehouse tienen objetivos distintos dentro de una organización.

Base de Datos Operacional (OLTP)

Las bases de datos operacionales están diseñadas para gestionar las operaciones cotidianas de una empresa. Permiten registrar, modificar y eliminar información de manera rápida y eficiente.

Características principales:

- Manejan transacciones en tiempo real.
- Almacenan información actualizada.
- Priorizan la velocidad de inserción y actualización.
- Son utilizadas por sistemas administrativos y operativos.

Data Warehouse (OLAP)

Los Data Warehouse están diseñados para el análisis de información y apoyo en la toma de decisiones.

Características principales:

- Conservan información histórica.
- Realizan consultas complejas y análisis multidimensionales.
- Integran información de distintas fuentes.
- Son utilizados por gerentes, analistas y directivos.

Base de Datos Operacional (OLTP)	Data Warehouse (OLAP)
Orientada a operaciones diarias	Orientada al análisis
Datos actuales	Datos históricos
Muchas inserciones y actualizaciones	Pocas modificaciones
Soporta transacciones	Soporta análisis y reportes
Consultas simples	Consultas complejas

Ventajas del Uso de un Data Warehouse

La implementación de un Data Warehouse proporciona numerosos beneficios a las organizaciones.

Integración de datos

Permite reunir información proveniente de diferentes sistemas y departamentos en un único repositorio centralizado.

Mejor toma de decisiones

Los directivos pueden acceder a información consolidada y actualizada para tomar decisiones fundamentadas.

Análisis histórico

Facilita el estudio de tendencias y comportamientos a lo largo del tiempo.

Mayor velocidad en consultas analíticas

Los datos están organizados específicamente para responder consultas complejas de manera eficiente.

Generación de reportes

Permite crear informes detallados sobre ventas, clientes, inventarios y otros indicadores clave de desempeño.

Incremento de la productividad

Reduce el tiempo requerido para buscar y analizar información.

Escalabilidad

Puede crecer conforme aumentan las necesidades de almacenamiento y análisis de la organización.

Arquitecturas Comunes de un Data Warehouse

Esquema Estrella (Star Schema)

Es la arquitectura más utilizada debido a su simplicidad y eficiencia.

Está formada por:

- Una tabla central de hechos.
- Varias tablas de dimensiones relacionadas directamente con la tabla de hechos.

Ventajas:

- Fácil comprensión.
- Consultas rápidas.
- Menor complejidad de diseño.

Ejemplo:

Tabla de hechos:

- Ventas

Tablas de dimensiones:

- Clientes
- Productos
- Tiempo
- Regiones

Esquema Copo de Nieve (Snowflake Schema)

Es una extensión del esquema estrella donde las dimensiones se encuentran normalizadas y divididas en varias tablas.

Ventajas:

- Reduce redundancia de datos.
- Mejor organización de la información.

Desventajas:

- Mayor complejidad.
- Consultas más lentas debido a la cantidad de relaciones.

Ejemplo:

Dimensión Cliente:

- Cliente
- Ciudad
- Estado
- País

Constelación de Hechos (Fact Constellation)

También conocida como esquema galaxia.

Contiene múltiples tablas de hechos que comparten algunas dimensiones.

Ventajas:

- Permite analizar diferentes procesos de negocio.
- Mayor flexibilidad.

Ejemplo:

Tabla de hechos:

- Ventas
- Compras
- Inventario

Dimensiones compartidas:

- Producto
- Tiempo
- Cliente

Herramientas y Tecnologías para Implementar un Data Warehouse

PostgreSQL

Es un sistema gestor de bases de datos relacional de código abierto ampliamente utilizado por su robustez, seguridad y rendimiento.

Características:

- Soporte para grandes volúmenes de datos.
- Compatibilidad con SQL estándar.
- Alta confiabilidad.

Python

Lenguaje de programación ampliamente utilizado para análisis de datos y procesos ETL.

Ventajas:

- Fácil de aprender.
- Gran cantidad de bibliotecas especializadas.
- Integración sencilla con bases de datos.

Pandas

Biblioteca de Python para manipulación y análisis de datos.

Funciones principales:

- Lectura de archivos CSV.
- Limpieza y transformación de datos.
- Procesamiento de grandes conjuntos de información.

SQLAlchemy

Herramienta que permite conectar aplicaciones Python con bases de datos relacionales mediante un enfoque orientado a objetos.

Jupyter Notebook

Entorno interactivo que facilita la ejecución de código Python, consultas SQL y documentación de resultados en un mismo documento.

Matplotlib

Biblioteca utilizada para crear gráficos estadísticos y visualizaciones de datos.

Permite generar:

- Gráficos de barras.

- Gráficos de líneas.
- Gráficos circulares.
- Histogramas.

Proceso ETL

El proceso ETL (Extract, Transform and Load) es fundamental en un Data Warehouse.

Sus etapas son:

1. Extracción (Extract): obtención de datos desde diversas fuentes.
2. Transformación (Transform): limpieza, validación y organización de los datos.
3. Carga (Load): almacenamiento de los datos transformados en el Data Warehouse.

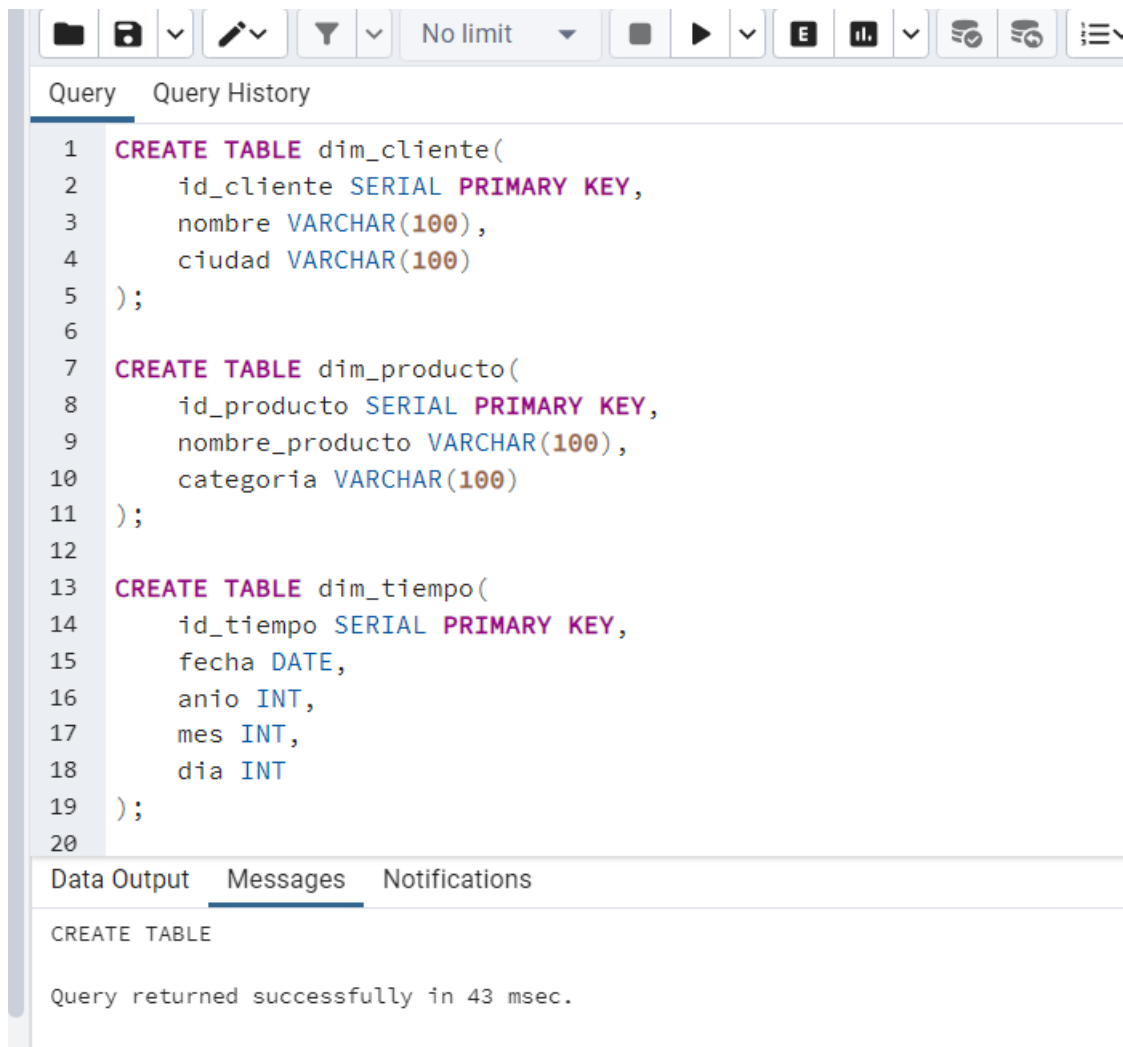
Este proceso garantiza la calidad y consistencia de la información utilizada para el análisis.

PRÁCTICA

Creación de la base de datos

✓ PostgreSQL 16/dw_ventas - Database connected. ✕

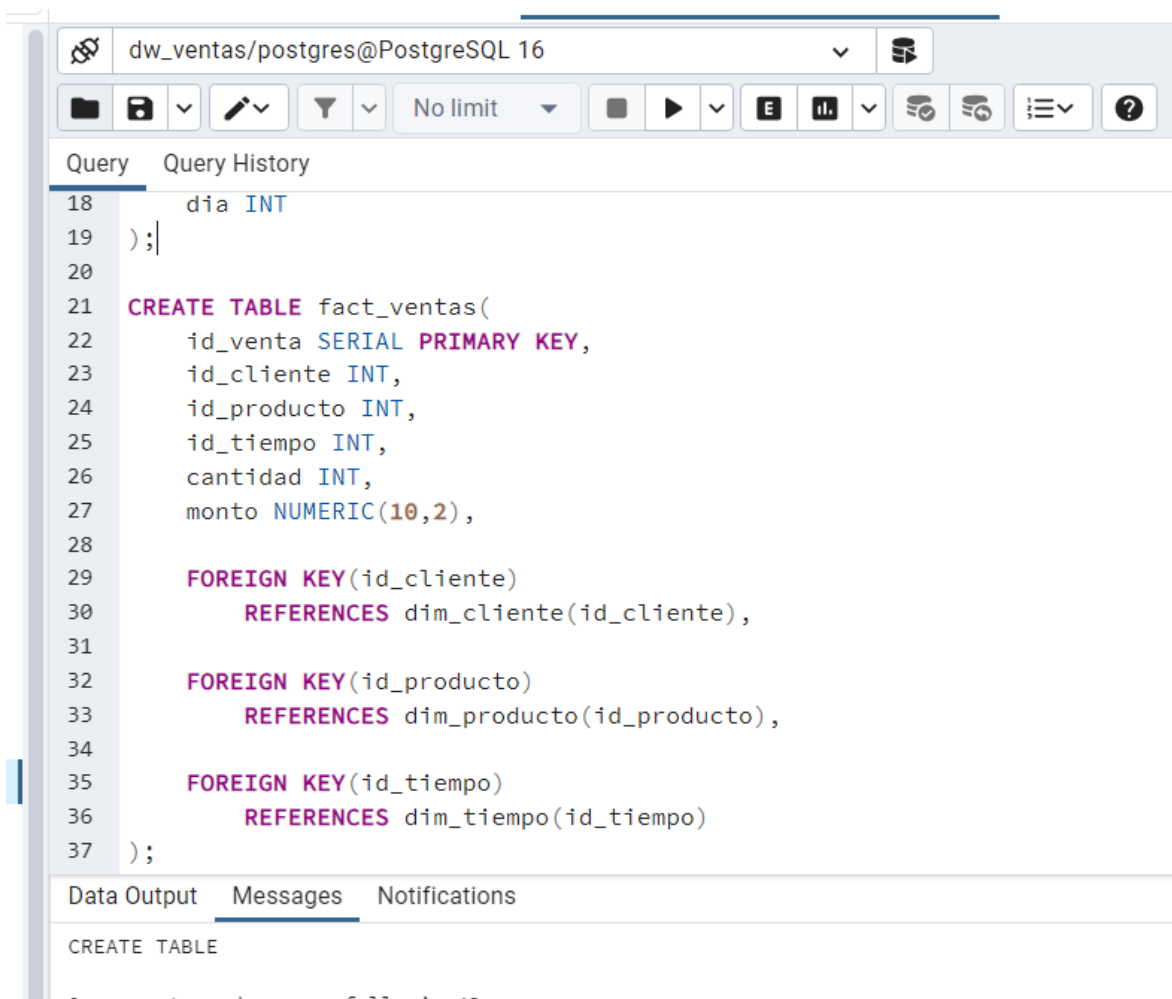
Tablas creadas correctamente



The screenshot shows a PostgreSQL query editor interface. At the top, there is a toolbar with icons for file operations, query execution, and settings. Below the toolbar, the 'Query' tab is active, displaying three SQL statements to create dimension tables: `dim_cliente`, `dim_producto`, and `dim_tiempo`. Each table has specific columns and data types, with primary keys defined. The 'Messages' tab at the bottom shows the confirmation message: 'Query returned successfully in 43 msec.'

```
1 CREATE TABLE dim_cliente(  
2     id_cliente SERIAL PRIMARY KEY,  
3     nombre VARCHAR(100),  
4     ciudad VARCHAR(100)  
5 );  
6  
7 CREATE TABLE dim_producto(  
8     id_producto SERIAL PRIMARY KEY,  
9     nombre_producto VARCHAR(100),  
10    categoria VARCHAR(100)  
11 );  
12  
13 CREATE TABLE dim_tiempo(  
14     id_tiempo SERIAL PRIMARY KEY,  
15     fecha DATE,  
16     anio INT,  
17     mes INT,  
18     dia INT  
19 );  
20
```

Query returned successfully in 43 msec.



```

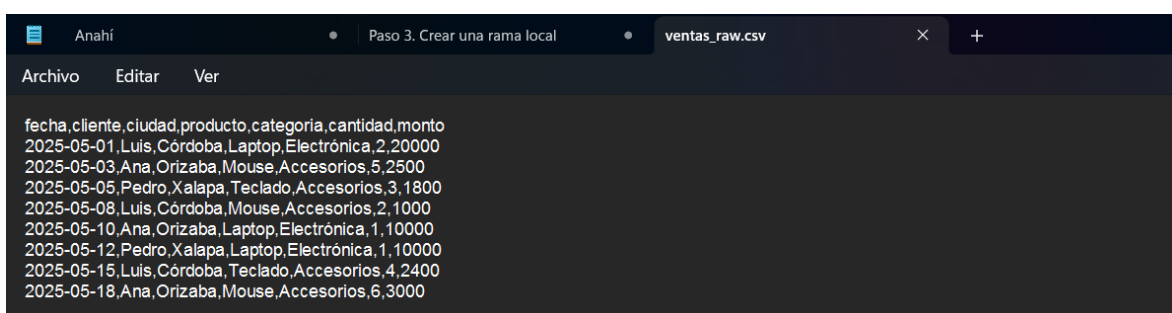
Query History
18      dia INT
19    );|
20
21  CREATE TABLE fact_ventas(
22      id_venta SERIAL PRIMARY KEY,
23      id_cliente INT,
24      id_producto INT,
25      id_tiempo INT,
26      cantidad INT,
27      monto NUMERIC(10,2),
28
29      FOREIGN KEY(id_cliente)
30          REFERENCES dim_cliente(id_cliente),
31
32      FOREIGN KEY(id_producto)
33          REFERENCES dim_producto(id_producto),
34
35      FOREIGN KEY(id_tiempo)
36          REFERENCES dim_tiempo(id_tiempo)
37  );
  
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 42 msec

Creación del archivo .csv



```

Anahí Paso 3. Crear una rama local ventas_raw.csv
Archivo Editar Ver
fecha,cliente,ciudad,producto,categoria,cantidad,monto
2025-05-01,Luis,Córdoba,Laptop,Electrónica,2,20000
2025-05-03,Ana,Orizaba,Mouse,Accesorios,5,2500
2025-05-05,Pedro,Xalapa,Teclado,Accesorios,3,1800
2025-05-08,Luis,Córdoba,Mouse,Accesorios,2,1000
2025-05-10,Ana,Orizaba,Laptop,Electrónica,1,10000
2025-05-12,Pedro,Xalapa,Laptop,Electrónica,1,10000
2025-05-15,Luis,Córdoba,Teclado,Accesorios,4,2400
2025-05-18,Ana,Orizaba,Mouse,Accesorios,6,3000
  
```

Se importan las librerías

jupyter DataWarehouse_Practica Last Checkpoint: 12 minutes ago

File Edit View Run Kernel Settings Help

Save + Copy Paste Run Cell Interrupt Code

```
[1]: import pandas as pd
      from sqlalchemy import create_engine
      import matplotlib.pyplot as plt
```

Conexión a PostgreSQL

Installing collected packages: psycopg2-binary
Successfully installed psycopg2-binary-2.9.12
You may need to restart the kernel to use updated packages.

Collapse Output

```
[4]: engine = create_engine(
      "postgresql://postgres:123456@localhost:5432/dw_ventas"
```

[]:

Lectura del CSV

```
[5]: df = pd.read_csv("../Data/ventas_raw.csv")
      df.head()
```

```
[5]:
```

	fecha	cliente	ciudad	producto	categoria	cantidad	monto
0	2025-05-01	Luis	Córdoba	Laptop	Electrónica	2	20000
1	2025-05-03	Ana	Orizaba	Mouse	Accesorios	5	2500
2	2025-05-05	Pedro	Xalapa	Teclado	Accesorios	3	1800
3	2025-05-08	Luis	Córdoba	Mouse	Accesorios	2	1000
4	2025-05-10	Ana	Orizaba	Laptop	Electrónica	1	10000

Creación de la dimensión cliente

```
[6]: dim_cliente = df[
      ["cliente", "ciudad"]
    ].drop_duplicates()

    dim_cliente.columns = [
        "nombre",
        "ciudad"
    ]

    dim_cliente
```

```
[6]:
```

	nombre	ciudad
0	Luis	Córdoba
1	Ana	Orizaba
2	Pedro	Xalapa

Creación de la dimensión producto

```
[7]: dim_producto = df[
      ["producto", "categoria"]
    ].drop_duplicates()

    dim_producto.columns = [
        "nombre_producto",
        "categoria"
    ]

    dim_producto
```

```
[7]:
```

	nombre_producto	categoria
0	Laptop	Electrónica
1	Mouse	Accesorios
2	Teclado	Accesorios

Creación de la dimensión tiempo

```
[8]: df["fecha"] = pd.to_datetime(df["fecha"])

dim_tiempo = pd.DataFrame()

dim_tiempo["fecha"] = df["fecha"]
dim_tiempo["anio"] = df["fecha"].dt.year
dim_tiempo["mes"] = df["fecha"].dt.month
dim_tiempo["dia"] = df["fecha"].dt.day

dim_tiempo = dim_tiempo.drop_duplicates()

dim_tiempo
```

```
[8]:
```

	fecha	anio	mes	dia
0	2025-05-01	2025	5	1
1	2025-05-03	2025	5	3
2	2025-05-05	2025	5	5
3	2025-05-08	2025	5	8
4	2025-05-10	2025	5	10
5	2025-05-12	2025	5	12
6	2025-05-15	2025	5	15
7	2025-05-18	2025	5	18

Se cargan las dimensiones

```
[9]: dim_cliente.to_sql(
      "dim_cliente",
      engine,
      if_exists="append",
      index=False
    )
```

```
[9]: 3
```

```
[ ]: |
```

```
[10]: dim_producto.to_sql(
      "dim_producto",
      engine,
      if_exists="append",
      index=False
    )
```

```
[10]: 3
```

```
[11]: dim_tiempo.to_sql(
      "dim_tiempo",
      engine,
      if_exists="append",
      index=False
    )
```

```
[11]: 8
```

Se crea la tabla de hechos

```
[12]: clientes_db = pd.read_sql(
      "SELECT * FROM dim_cliente",
      engine
    )

    productos_db = pd.read_sql(
      "SELECT * FROM dim_producto",
      engine
    )

    tiempo_db = pd.read_sql(
      "SELECT * FROM dim_tiempo",
      engine
    )
```

```
[13]: fact = df.merge(
      clientes_db,
      left_on=["cliente","ciudad"],
      right_on=["nombre","ciudad"]
    )
```

```
[14]: fact = fact.merge(
      productos_db,
      left_on=["producto","categoria"],
      right_on=["nombre_producto","categoria"]
    )
```

```
[18]: fact = fact.merge(
      tiempo_db,
      on="fecha"
    )
```

```
[ ]:
```

```
[21]: fact_ventas = fact[
      [
        "id_cliente",
        "id_producto",
        "id_tiempo",
        "cantidad",
        "monto"
      ]
    ]
```

Se carga la tabla de hechos

```
[22]: fact_ventas.to_sql(
      "fact_ventas",
      engine,
      if_exists="append",
      index=False
    )
```

[22]: 8

Primera consulta

```
[23]: consulta = """
      SELECT
        p.nombre_producto,
        SUM(f.cantidad) total_vendido
      FROM fact_ventas f
      JOIN dim_producto p
      ON f.id_producto = p.id_producto
      GROUP BY p.nombre_producto
      """

      resultado = pd.read_sql(
        consulta,
        engine
      )

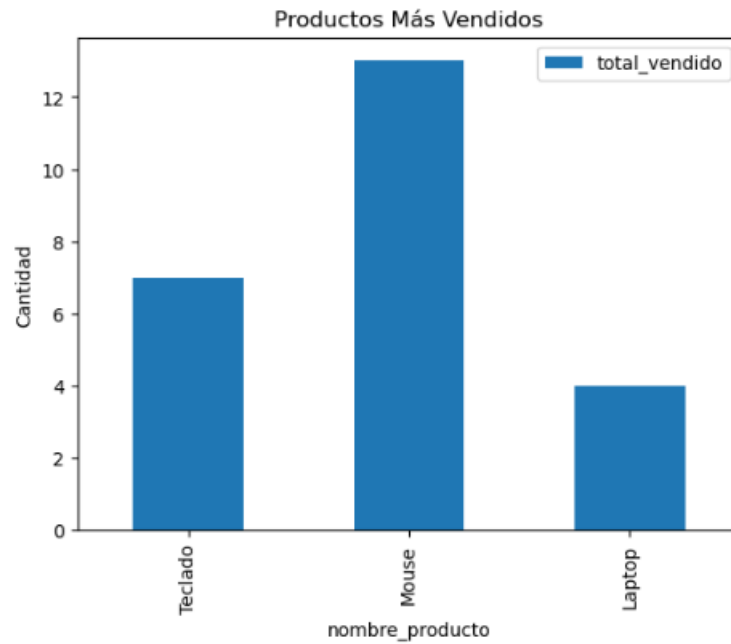
      resultado
```

```
[23]:
```

	nombre_producto	total_vendido
0	Teclado	7
1	Mouse	13
2	Laptop	4

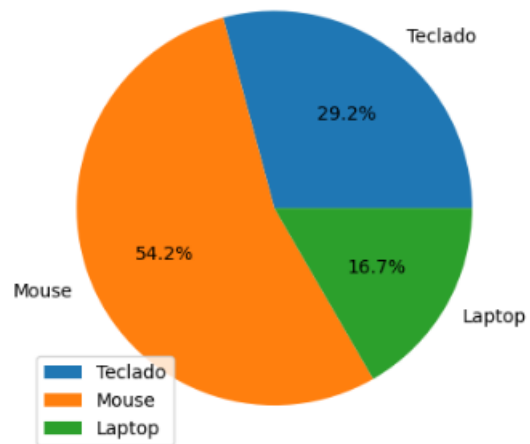
Gráfica de barras

```
[24]: resultado.plot(  
    kind="bar",  
    x="nombre_producto",  
    y="total_vendido"  
)  
  
plt.title("Productos Más Vendidos")  
plt.ylabel("Cantidad")  
plt.show()
```



Gráfica circular

```
[25]: resultado.set_index(  
        "nombre_producto"  
    ).plot(  
        kind="pie",  
        y="total_vendido",  
        autopct="%1.1f%%"  
    )  
  
plt.ylabel("")  
plt.show()
```



```
[ ]:
```